SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

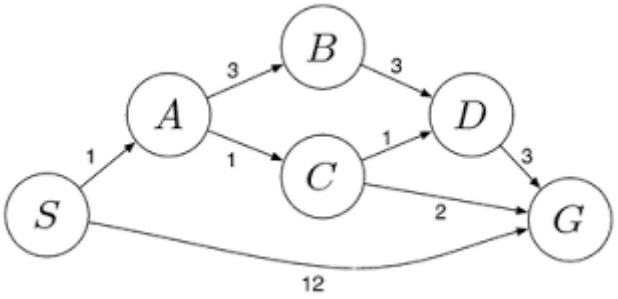
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

		requirements accurately			

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — ANALYTICAL PROBLEM SOLVING
Course: Artificial Intelligence | Course Code: CSA17

Name: SARAVANAN.T **Register No.: 1925260349**

1. Water Jug Problem

Given: One 4-gallon jug and one 3-gallon jug, with no measuring marks. The aim is to obtain exactly 2 gallons in the 4-gallon jug.

Step	Action	4-gallon jug	3-gallon jug
1	Fill the 3-gallon jug from the pump.	0	3
2	Pour the 3-gallon jug into the 4-gallon jug.	3	0
3	Fill the 3-gallon jug again.	3	3
4	Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug	is4full. Only 1 gallo	n2fits.
5	Empty the 4-gallon jug onto the ground.	0	2
6	Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon	ju2g.	0

Result: The 4-gallon jug contains exactly 2 gallons. This solution uses only filling, pouring and emptying, as allowed in the question.

2. Mars Rover as an Intelligent Agent

i. Percepts: The rover receives information through cameras and other sensors, such as images of the terrain, rocks and soil; distance to obstacles; slope and surface conditions; location and orientation; temperature and equipment status; and readings from scientific instruments analysing samples.

ii. Operating environment: Mars is a physical, partially observable, uncertain, dynamic and sequential environment. The rover cannot observe everything at once, terrain and weather can create hazards, actions take time, and energy and communication are limited. It is mostly a single-agent environment, although Earth-based operators also provide commands.

iii. Actions: Move or stop; turn and adjust speed; aim and operate cameras; select rocks or soil for investigation; use a robotic arm or sampling tools; perform scientific measurements; store and transmit data; manage power; and enter a safe mode when necessary.

iv. Performance evaluation: Evaluate the rover by the quality and quantity of useful scientific data collected, accuracy of sample analysis, successful completion of mission goals, distance safely travelled, efficient use of energy and time, avoidance of damage, and reliability of communication. These measures should be considered together.

v. Suitable agent architecture: A model-based, goal-based agent with utility-based decision-making is suitable. Its internal model helps track location and unseen surroundings; its goals guide navigation and sample collection; and a utility measure helps choose safer, more informative actions while balancing time and energy. A learning component can improve decisions using experience.

3. Eight-Queens Problem: Search-Space Formulation

Goal: Place eight queens on an 8 × 8 chessboard so that no two queens attack each other. Queens must not share a row, column or diagonal.

Problem components:

• **Initial state:** An empty chessboard.
• **State representation:** A sequence of row positions, one for each column. For example, [1, 5, 8, 6, 3, 7, 2, 4] means the queen in column 1 is in row 1, the queen in column 2 is in row 5, and so on.
• **Actions:** Place a queen in a safe square in the next column, or move a queen in a formulation that permits rearrangement.
• **Transition model:** Add the selected queen to the board and update the configuration.
• **Goal test:** Eight queens are placed and no pair shares a row, column or diagonal.
• **Path cost:** Commonly, each placement has cost 1; the main aim is to find a valid solution.

Search strategy: Backtracking depth-first search is effective: place one queen per column, try safe rows, and backtrack whenever no safe row remains. Breadth-first or other systematic search methods can also explore states, but may use more memory.

One valid solution: [1, 5, 8, 6, 3, 7, 2, 4]. Each number gives the row for columns 1 through 8. No two queens in this configuration share a row, column or diagonal.

4. OLA Cab Booking: Problem-Solving Agent

Suitable agent: A goal-based problem-solving agent, supported by cost/utility evaluation. The goal is to reach the customer's destination. The agent considers available cab types (Mini, Micro, Sedan, Shared, Prime, etc.) and compares estimated fare, arrival time, comfort, availability and the customer's preferences before recommending or booking a suitable cab.

Problem formulation:
• Initial state: Customer's current pickup location and booking preferences.
• Actions: Select pickup and destination, request available cab types, compare options, select a cab and confirm booking.
• Goal state: A suitable cab is booked to take the customer to the destination.
• Evaluation: Total fare, waiting time, travel time, comfort, availability and preference match.

Pseudocode:

```
BEGIN
  READ pickup_location, destination
  READ customer_preferences
  options ← GET_AVAILABLE_CABS(pickup_location, destination)

  IF options is empty THEN
     DISPLAY "No cabs available; try again later"
     STOP
  END IF

  FOR each cab in options DO
     ESTIMATE fare, pickup_time, travel_time
     CHECK comfort, cab_type and preference_match
     SCORE cab using the customer's priorities
  END FOR

  best_cab ← option with the best score
  DISPLAY best_cab, estimated_fare, pickup_time
  IF customer confirms THEN
     BOOK(best_cab)
     DISPLAY booking_confirmation
  ELSE
```

```
    DISPLAY "Booking cancelled"
  END IF
END
```

5. Uniform Cost Search (UCS) for Warehouse Delivery

Objective: Find a path from starting warehouse S to goal warehouse G with the minimum total route cost. Each edge weight represents the route's travel cost (fuel plus time).

Uniform Cost Search procedure:

1. Put the start node S into a priority queue with cumulative cost 0.
2. Remove the node with the smallest cumulative cost from the queue.
3. If that node is G, return its path and total cost.
4. Otherwise, examine each neighbouring warehouse. For every route, calculate new_cost = cost_to_current + edge_cost.
5. If the neighbour has not been reached before, or this route is cheaper than its best known cost, update its cost and parent and insert/update it in the priority queue.
6. Repeat until G is removed from the queue. If the queue becomes empty first, no route exists.

Pseudocode:

```
UCS(start, goal):
   frontier ← priority queue ordered by path_cost
   push(frontier, (cost=0, node=start, path=[start]))
   best_cost[start] ← 0

   while frontier is not empty:
      (cost, node, path) ← pop_min(frontier)

      if node = goal:
         return path, cost

      if cost > best_cost[node]:
         continue

      for each (neighbor, edge_cost) of node:
         new_cost ← cost + edge_cost
         if neighbor not in best_cost OR new_cost < best_cost[neighbor]:
            best_cost[neighbor] ← new_cost
            push(frontier, (new_cost, neighbor, path + [neighbor]))

   return "No path exists
```

Important note about the supplied question: The PDF text says that the delivery network is shown as a weighted graph, but the actual graph and its edge weights are not visible in the uploaded document. Therefore, the exact least-cost path and numerical total cost from S to G cannot be calculated reliably from the supplied file alone. Apply the steps above to the graph provided by your instructor.

End of answers